



Manual Técnico

PROYECTO 2

3350910240901 - Karla Fernanda Matías de León

3190701150910 - Hania Mirleth Mazariegos Alonzo



Tecnologías utilizadas

El proyecto hizo uso de una arquitectura modular basada en contenedores y servicios web con el objetivo de construir una aplicación escalable y monitoreable. A continuación, se describen las principales tecnologías utilizadas en el sistema:

Redis

Redis es una base de datos en memoria, de tipo clave-valor, utilizada por su rapidez. En este proyecto, Redis se utiliza como sistema de almacenamiento principal para las entidades:



redis

- Usuarios
- Videojuegos
- Reseñas

Los principales beneficios que redis ofrece para este sistema son:

- Respuestas rápidas (baja latencia)
- Modelo simple basado en estructuras de datos como hashes y listas
- Persistencia opcional
- Integración con exportadores para métricas

Node.js

Node.js es un entorno de ejecución para JavaScript del lado del servidor.

En este sistema se usa para construir el backend de la aplicación, que expone una API REST que permite:

- Autenticación de usuarios
- Gestión de videojuegos
- Envío, consulta y eliminación de reseñas

Algunas de las características destacadas que ofrece node son:

- Modelo asíncrono y no bloqueante
- Rápida integración con Redis a través de bibliotecas
- Excelente rendimiento para aplicaciones I/O intensivas



React

React es una biblioteca de JavaScript para la construcción de interfaces de usuario. En este proyecto se utilizó para desarrollar el frontend de modo que la interacción con la aplicación sea dinámica y reactiva.



React facilita la separación de componentes, el manejo del estado con `useState` y `useContext`, y la navegación con `react-router-dom`.

Funciones implementadas con React:

- Inicio de sesión y validación
- Panel de administración y usuario final
- Visualización de videojuegos y reseñas
- Interacción mediante modales y menús

Prometheus

Prometheus es un sistema de monitoreo y alerta diseñado para recopilar métricas de servicios y aplicaciones.



Prometheus hace scraping periódico a los endpoints de métricas y los almacena en su propia base de datos interna. En este proyecto, se utiliza para:

- Recolectar métricas del contenedor de Redis mediante el redis_exporter
- Consultar datos históricos de uso y rendimiento
- Servir de fuente de datos para Grafana

Grafana

Grafana es una plataforma de visualización de datos y monitoreo. Se conecta fácilmente a Prometheus como fuente de datos y permite construir paneles interactivos

En el sistema se utiliza para:

- Visualizar métricas recolectadas por Prometheus
- Crear dashboards personalizados sobre Redis
- Supervisar el estado del sistema



Docker

Docker permite empaquetar y ejecutar aplicaciones en contenedores ligeros y portables. Toda la arquitectura del proyecto (excepto el frontend) está dockerizada y orquestada mediante docker-compose.



Servicios definidos:

- **redis**: base de datos principal
- **backend**: API Node.js
- **redis_exporter**: exportador de métricas para Redis
- **prometheus**: sistema de monitoreo
- **grafana**: panel de visualización

Docker permite que cada componente funcione de forma aislada y sea replicable en otros entornos.



Arquitectura del sistema

La arquitectura del sistema se basa en una estructura cliente-servidor desacoplada, complementada con contenedores Docker para facilitar el despliegue y el monitoreo del estado del sistema.

Módulos principales:

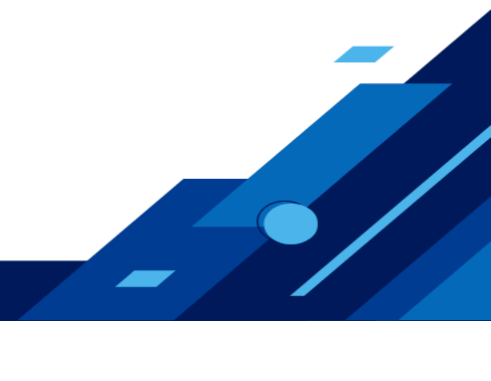
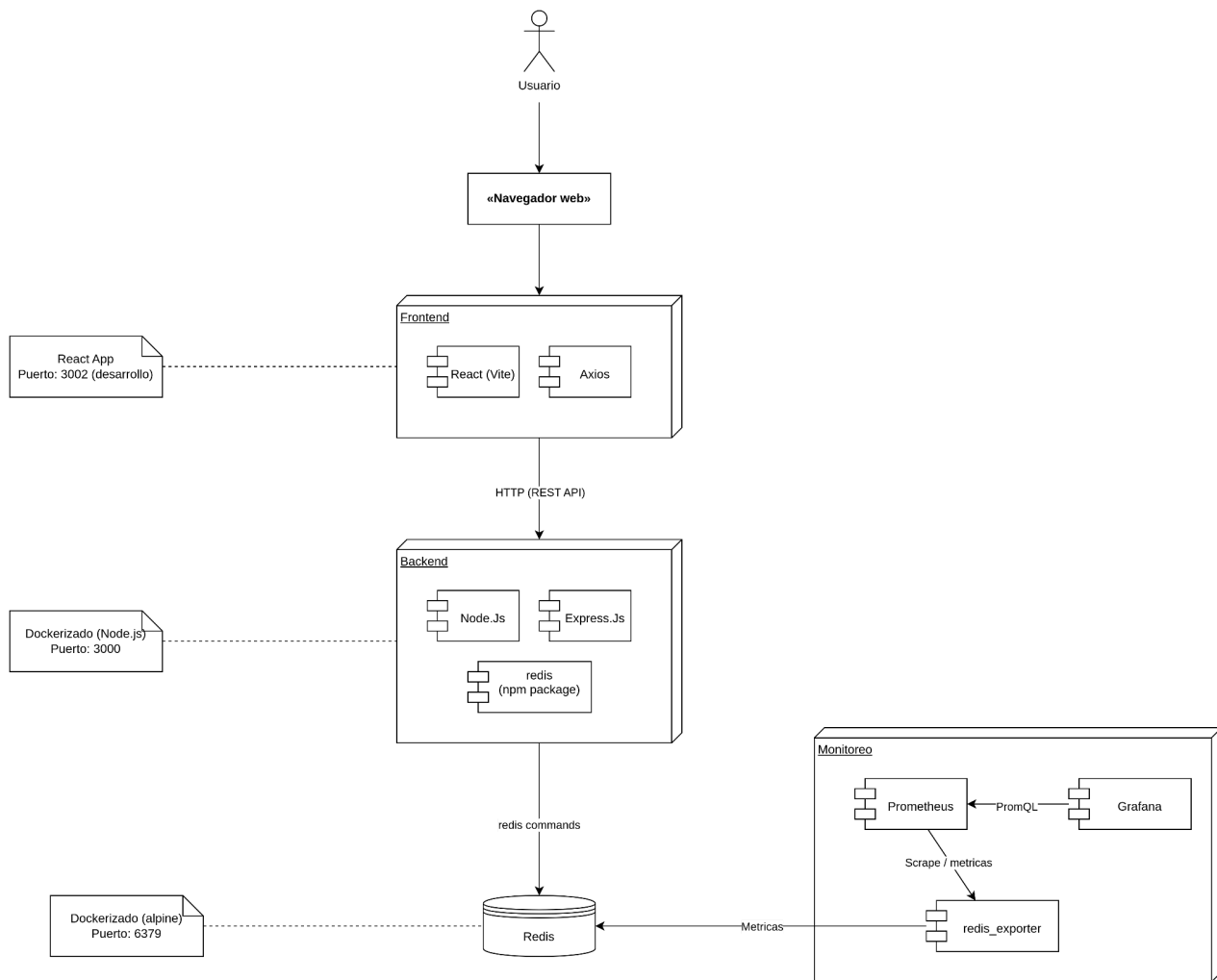
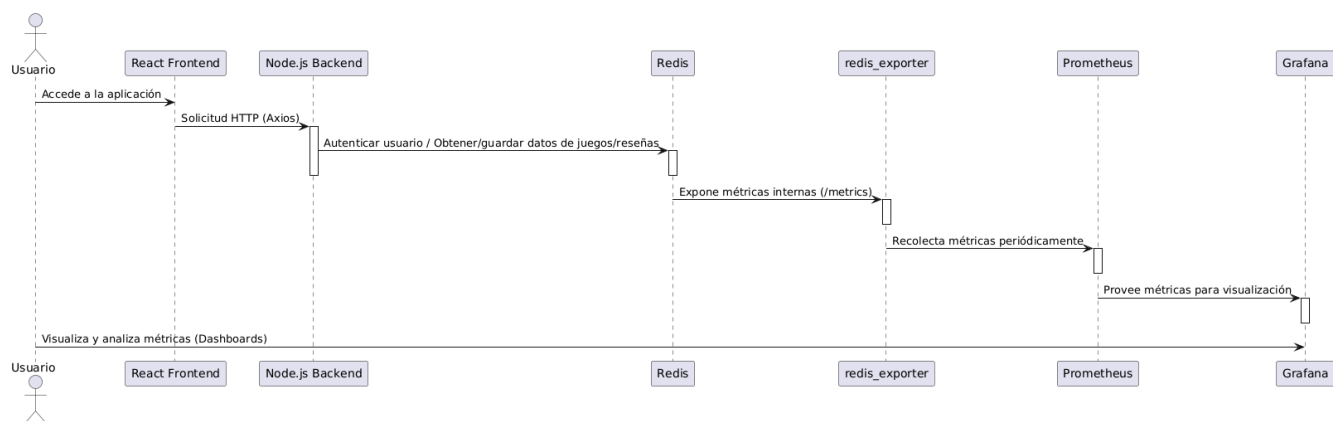
- **frontend (React):** Interfaz del usuario que consume una API REST para las operaciones de login, listado de juegos, publicación de reseñas, etc.
 - **Backend** (Node.js + Express): API REST que conecta con la base de datos Redis, valida usuarios y expone endpoints.
 - **Redis:** Base de datos principal, almacena toda la información sobre usuarios, juegos y reseñas.
 - **Prometheus:** Sistema de monitoreo que recopila métricas de redis de forma periódica.
 - **redis_exporter:** Expone métricas internas de Redis para ser recolectadas por Prometheus.
 - **Grafana:** Plataforma de visualización que consulta a Prometheus y muestra dashboards para el monitoreo del sistema.
- 

Diagrama de arquitectura



Flujo de datos





Requerimientos del sistema

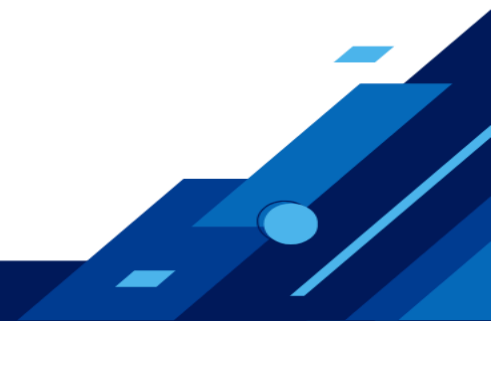
Para ejecutar esta aplicación de manera correcta, se deben cumplir ciertos requerimientos a nivel de hardware, sistema operativo, software y versiones mínimas de las tecnologías involucradas.

Requisitos de hardware

Componente	Requisito mínimo	Recomendado
CPU	2 núcleos	4 núcleos o más
Memoria RAM	4 GB	8 GB o más
Almacenamiento	2 GB libres	10 GB SSD
Conectividad	Red local o internet disponible	Conexión estable a internet

Requisitos de software

Software	Requisito
Sistema Operativo	Linux (Ubuntu 20.04+), macOS, Windows 10+
Docker	v20.10.0 o superior
Docker Compose	v2.0.0 o superior
Node.js (desarrollo del backend)	v18.0.0 o superior
npm (Node Package Manager)	v8.0.0 o superior



Versiones mínimas de las tecnologías utilizadas

Tecnología	Versión mínima recomendada	Observaciones
Node.js	18.0.0	Usado para el servidor backend
npm	8.0.0	Para instalar dependencias de Node y React
Redis	6.0.0	Base de datos principal, usada en contenedor
React	18.0.0	Usado en el frontend, versión moderna con hooks
Prometheus	2.45.0	Monitoreo, scrape de métricas de Redis
Grafana	10.0.0	Visualización de métricas
redis_exporter	1.55.0	Exportador compatible con Redis
Docker	20.10.2000	Contenerización de servicios
docker-compose	2.0.0	Orquestación de servicios

Navegadores compatibles

El frontend está diseñado para funcionar correctamente en los navegadores:

- Google Chrome (v90 o superior)
- Firefox (v90 o superior)
- Microsoft Edge (v90 o superior)

Procedimiento de instalación y despliegue

A continuación se describen los pasos necesarios para levantar y ejecutar la aplicación, tanto con Docker como con el frontend manualmente. Se asume que ya se tienen instalados Docker, Docker Compose, Node.js y npm

1. Dirigirse al directorio del proyecto:

```
$ cd <ruta al directorio>/BD2-EVJ2025_-12/Proyecto2
```

2. Levantar el proyecto con docker:

- 2.1. Dirigirse al directorio de backend:

```
$ cd backend
```

- 2.2. Verificar el contenido del **docker-compose.yml**, el archivo contendrá los siguientes servicios:

- redis
- backend
- prometheus
- grafana
- redis-exporter.

- 2.3. Construir e iniciar los servicios:

```
$ docker-compose up --build -d
```



Esto hará lo siguiente:

- Construye el contenedor del backend (desde backend/Dockerfile)
- Levanta Redis en el puerto 6379
- Levanta Prometheus en el puerto 9090
- Levanta Grafana en el puerto 3001
- Levanta redis_exporter en el puerto 9121

2.4. Se puede ver el estado de los contenedores con:

```
$ docker ps
```

2.5. Verificar los servicios:

- Backend: <http://localhost:3000/>
- Prometheus: <http://localhost:9090/>
- Grafana: <http://localhost:3001/> (usuario por defecto: admin / admin)

3. Levantar el frontend manualmente

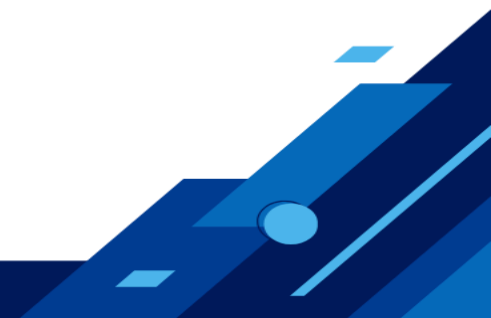
3.1. Ir al directorio del frontend:

```
$ cd ../fronted/frontend
```

3.2. Instalar dependencias:

```
$ npm install
```

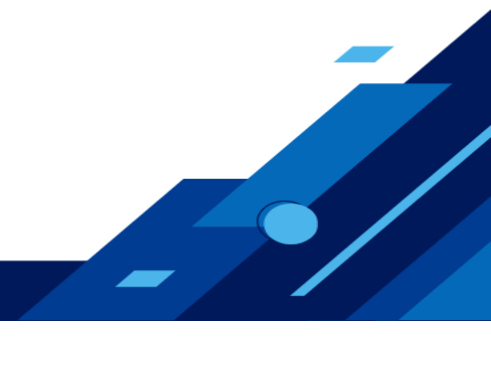
3.3. Iniciar el servidor:





```
$ npm start
```

Esto abrirá la aplicación en <http://localhost:3002/> desde el navegador ya se podrá interactuar con el sistema.



Endpoints

Crear usuario

Solicitud: POST /users

Descripción: Crea un nuevo usuario en el sistema.

Parámetros del cuerpo (JSON):

```
{  
  "username": "juan",  
  "email": "juan@example.com",  
  "password_hash": "clave123",  
  "role": "user"  
}
```

Redis utilizado:

```
HSET user:<uuid> username "juan" email "juan@example.com"  
password_hash "<bcrypt_hash>" role "user"
```



Listar usuarios

Solicitud: GET /users

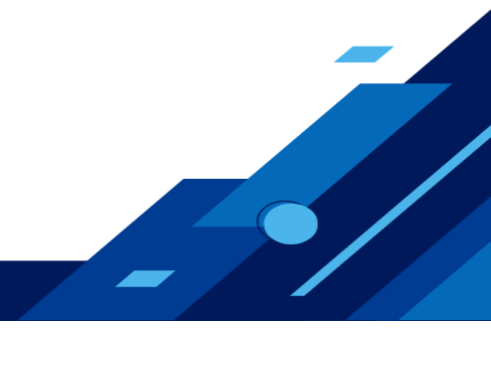
Descripción: Devuelve todos los usuarios registrados.

Redis utilizado:

```
KEYS user:*  
HGETALL user:<id>
```

Ejemplo respuesta (JSON):

```
[  
  {  
    "id": "c4f9e8c3",  
    "username": "juan",  
    "email": "juan@example.com",  
    "password_hash": "$2b$10$...",  
    "role": "user"  
  },  
  ...  
]
```





Eliminar usuario

Solicitud: DELETE /users/:id

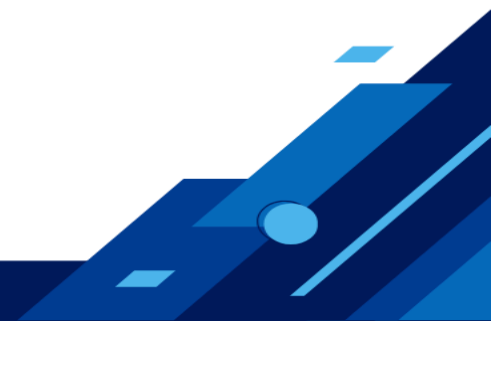
Descripción: Elimina un usuario existente por su ID.

Redis utilizado:

```
EXISTS user:<id>  
DEL user:<id>
```

Respuesta exitosa (JSON):

```
{  
  "message": "Usuario con ID <id> eliminado"  
}
```



Crear juego

Solicitud: POST /games

Descripción: Crea un nuevo juego en la base de datos.

Parámetros del cuerpo (JSON):

```
{  
  "titulo": "Elden Ring",  
  "descripcion": "Un juego de acción y aventura en mundo abierto.",  
  "genero": "RPG",  
  "desarrollador": "FromSoftware",  
  "plataformas": "PC, PS5, Xbox",  
  "fecha_lanzamiento": "2022-02-25",  
  "clasificacion_edad": "M",  
  "imagen_url": "https://example.com/eldenring.jpg"  
}
```

Redis utilizado:

```
HSET game:<uuid> titulo "Elden Ring" descripcion "..." genero "RPG"  
desarrollador "FromSoftware" plataformas "PC, PS5, Xbox"  
fecha_lanzamiento "2022-02-25" clasificacion_edad "M" imagen_url  
"https://..."
```

Listar juegos

Solicitud: GET /games

Descripción: Devuelve todos los usuarios registrados.

Redis utilizado:

```
KEYS game:*
HGETALL game:<id>
KEYS review:*
HGETALL review:<id> (para cada reseña relacionada al juego)
```

Ejemplo respuesta (JSON):

```
[
  {
    "id": "1f23a4bd",
    "titulo": "Elden Ring",
    "descripcion": "...",
    "genero": "RPG",
    "desarrollador": "FromSoftware",
    "plataformas": "PC, PS5",
    "fecha_lanzamiento": "2022-02-25",
    "clasificacion_edad": "M",
    "imagen_url": "...",
  }, ...
]
```



Eliminar juego

Solicitud: DELETE /games/:id

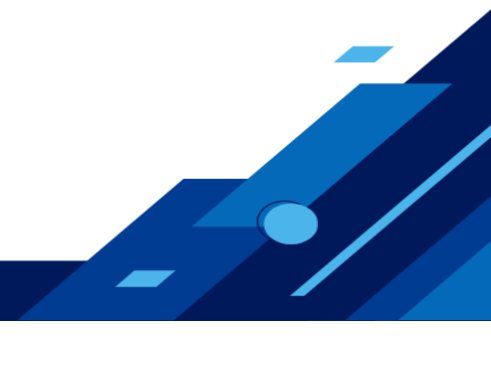
Descripción: Elimina un juego existente por su ID.

Redis utilizado:

```
EXISTS game:<id>  
DEL game:<id>
```

Respuesta exitosa (JSON):

```
{  
  "message": "Juego con ID <id> eliminado"  
}
```





Login de usuario

Solicitud: POST /login

Descripción: Valida las credenciales del usuario y retorna su información básica si las credenciales son válidas.

Cuerpo esperado (JSON):

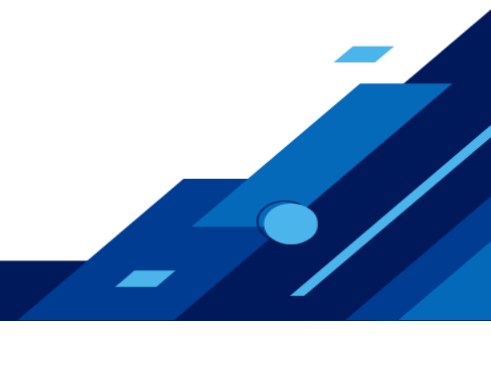
```
{  
  "email": "juan@example.com",  
  "password": "clave123"  
}
```

Redis utilizado:

```
KEYS user:*  
HGETALL user:<id>
```

Respuesta exitosa (JSON):

```
{  
  "id": "c4f9e8c3",  
  "username": "juan",  
  "role": "user"  
}
```



Crear reseña

Solicitud: POST /reviews

Descripción: Crea o sobrescribe una nueva reseña para un videojuego.

Parámetros del cuerpo (JSON):

```
{
  "game_id": "1f23a4bd",
  "user_id": "c4f9e8c3",
  "score": 4.5,
  "comment": "Muy buen juego",
  "overwrite": true
}
```

Comportamiento condicional:

Si el usuario ya hizo una reseña para ese juego:

- Con "overwrite": false → error 409
- Con "overwrite": true → la reseña anterior se elimina

Redis utilizado:

```
EXISTS game:1f23a4bd
EXISTS user:c4f9e8c3
```

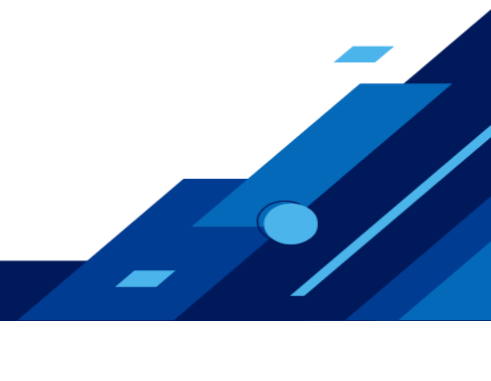


```
KEYS review:*           (para buscar si existe una reseña del
usuario sobre ese juego)
DEL review:<old_id>       (si existe y se sobrescribe)
HSET review:<uuid> game_id "..." user_id "..." score "..." comment
"..." timestamp "..."
```

Respuesta exitosa:

```
{
  "message": "Reseña agregada",
  "review_id": "review-uuid"
}
```

Errores posibles:

- 400 Faltan datos
 - 404 Juego o usuario no encontrado
 - 409 Ya existe reseña y no se permitió sobrescribir
- 



Eliminar reseña

Solicitud: DELETE /review/:id

Descripción: Elimina una reseña específica por su ID.

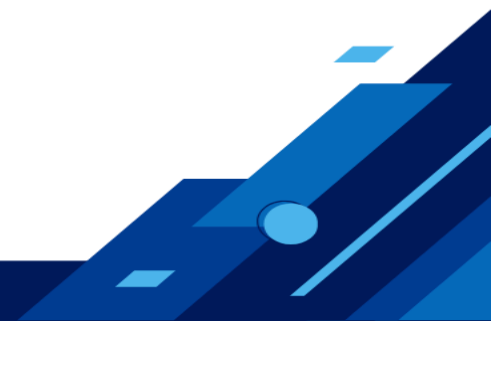
Redis utilizado:

```
EXISTS review:<id>  
DEL review:<id>
```

Respuesta exitosa (JSON):

```
{  
  "message": "Reseña con ID <id> eliminado"  
}
```

Errores posibles:

- 404 Reseña no encontrada
- 

Consultar reseñas por juego

Solicitud: GET /games/:id/reviews

Descripción: Recupera todas las reseñas que han sido hechas para un videojuego específico.

Redis utilizado:

```
KEYS review:*  
HGETALL review:<id>           -- por cada reseña  
HGETALL user:<review.user_id> -- para obtener el username
```

Respuesta exitosa (JSON):

```
[  
  {  
    "id": "r1",  
    "username": "gamer123",  
    "game_id": "1f23a4bd",  
    "user_id": "u1",  
    "score": "4.5",  
    "comment": "Excelente jugabilidad",  
    "timestamp": "1719264600000"  
  }, ...  
]
```

Consultar reseñas por usuario

Solicitud: GET /users/:id/reviews

Descripción: Recupera todas las reseñas realizadas por un usuario específico.

Redis utilizado:

```
EXISTS user:u1
KEYS review:*
HGETALL review:<id>          -- para revisar si pertenece al usuario
HGETALL game:<review.game_id> -- para obtener el título del juego
```

Respuesta exitosa (JSON):

```
[
  {
    "id": "r1",
    "game_id": "1f23a4bd",
    "user_id": "u1",
    "score": "4.5",
    "comment": "Excelente jugabilidad",
    "timestamp": "1719264600000",
    "game_title": "Halo Infinite"
  }, ...
]
```



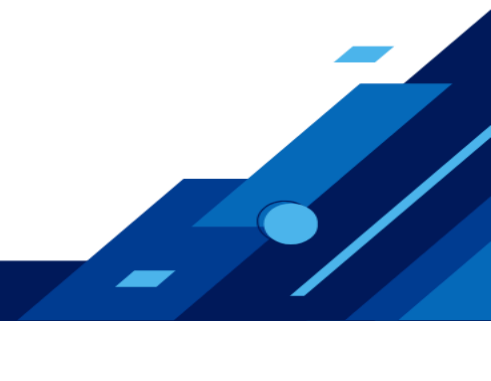
Análisis de resultados

El desarrollo de aplicaciones modernas exige estructuras de procesamiento y almacenamiento de datos eficientes al mismo tiempo que requieren de herramientas adecuadas de monitoreo del sistema. La elección de tecnologías como Redis, Prometheus y Grafana para este sistema de reseñas de videojuegos aporta rendimiento y monitoreo efectivo.

Redis, una base de datos NoSQL basada en estructuras de datos en memoria, en este caso es el motor principal de almacenamiento de la aplicación. Su arquitectura y su operación en memoria garantizan velocidad al manipular datos, esto le permite a la aplicación responder rápidamente a cada interacción del usuario.

Además de su rendimiento, redis también es muy flexible. A diferencia de las bases de datos relacionales, no requiere esquemas predefinidos, por ello se puede modelar con facilidad las entidades necesarias a través de estructuras Hash, esto simplificó el desarrollo inicial, y también facilita la adaptación de la base de datos ante cambios en los requisitos del proyecto.

La capacidad de Redis para persistir datos en disco, aun siendo una base en memoria, garantiza que la información del sistema se mantenga disponible. Así, gracias a Redis el sistema ofrece una combinación entre rendimiento y durabilidad.





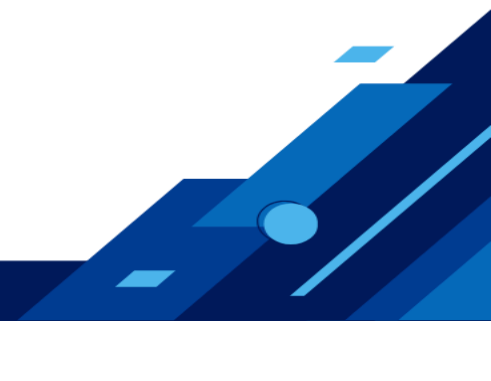
Por otro lado, ya que se busca un sistema mantenible es necesario contar con herramientas de monitoreo que permitan identificar errores, cuellos de botella o patrones de uso. Es por ello que se involucra el uso de Prometheus, como solución de monitorización basada en métricas temporales, por sus capacidades para recolectar, almacenar y consultar métricas personalizadas.

Por medio de la integración de Prometheus con el backend se expusieron métricas como contadores de usuarios registrados, juegos añadidos o reseñas creadas, entre otras.

Un aspecto valioso de Prometheus es su capacidad de operar por sí mismo, sin agentes externos, esto es porque accede a los endpoints de métricas y extrae los datos mediante scrape y con ello reduce la complejidad de configuración y permite una integración fluida con otros proyectos en contenedores mediante Docker.

Prometheus gestiona la recolección de métricas, Grafana las convierte en información útil y comprensible. Grafana es una plataforma de visualización de datos que permite construir paneles interactivos con las métricas recolectadas, proporcionando una interfaz clara y personalizable para la supervisión de sistemas.

En el contexto de este proyecto, Grafana se utilizó para visualizar estadísticas como la actividad del sistema y el comportamiento de los usuarios. El uso de dashboards





facilita la supervisión diaria del estado del backend y también son herramientas efectivas para presentaciones o informes de gestión.

La combinación de Redis, Prometheus y Grafana ha permitido que se desarrolle una aplicación con un backend eficiente, escalable y observable. Redis garantiza el almacenamiento rápido y flexible de los datos; Prometheus ofrece una capa de monitoreo robusta y personalizable; y Grafana permite transformar esas métricas en información visual que guía las decisiones de operación y mejora.

